

Weight Buddy

Product Requirements Document

A personal voice calorie, macro & exercise tracker. Speak what you ate or how you moved — parsed by your own OpenAI key, stored entirely on your device.

VERSION **1.0.4 (build 4)** · DATE **31 Aug 2026**
PLATFORM **Flutter — iOS / Android** · APP ID **com.tedif.weight_buddy**
STATUS **Shipped — v1.0.x maintenance + v1.1 scope**

1. Overview

Weight Buddy is a single-user, offline-first mobile app for tracking calories, macronutrients and exercise by **speaking instead of searching**. The user holds a button, says "two plates of jollof rice with two pieces of fried chicken", and gets a reviewable breakdown of items, calories and macros in a few seconds.

It differs from mainstream trackers in three deliberate ways:

- **Regional food is a first-class citizen.** Audio is transcribed by OpenAI's audio API with a user-supplied dish-name vocabulary, not by the device's native recognizer, so *egusi*, *amala*, *suya* and *dodo* are heard correctly. Portions are estimated from natural phrasing ("two plates"), not from a food database the user must search.
- **No backend, no subscription, no account.** Bring-your-own OpenAI API key. All data lives in on-device SQLite; the key lives in the platform secure vault. The only outbound traffic is to `api.openai.com`.
- **The model parses; it never prices exercise.** Workout burns are computed by a deterministic local engine (mechanical work for reps, MET for duration) off the user's latest weigh-in — so a 20-rep set logs ~7 kcal, not an invented workout-sized number.

1.1 Problem

Calorie tracking fails for two compounding reasons. First, **logging friction**: searching a food database, picking a match, then setting a portion takes 30–60 seconds per item, several times a day, and adherence collapses within weeks. Second, **database coverage**: users outside North America and Western Europe routinely cannot find their actual meals, and settle for a wrong "closest match" that quietly destroys the data's usefulness. Voice input on native phone recognizers does not solve this — those engines mangle regional dish names into unrelated English words.

1.2 Goals

GOAL	WHAT IT MEANS CONCRETELY
Frictionless logging	A meal is logged in one hold-to-speak gesture plus one confirm tap. Target: under 15 seconds end-to-end.
Accuracy on local foods	Regional dish names transcribe correctly via OpenAI audio + custom vocabulary hints.
Honest numbers	Exercise burn is computed deterministically, never guessed by the LLM. Estimates are never fabricated when inputs are missing.
Full macro picture	Protein / carbs / fat tracked alongside calories, with no extra user effort.
Privacy & ownership	Zero cloud backend. Data is on-device, exportable, and deletable in one action.
Sustained adherence	Streaks, month calendar, weight trend and a coach chat give the user reasons to return.

1.3 Non-goals

- **Multi-user accounts, social features, or sharing.** This is a single-owner app; there is no login.
- **A hosted backend or a managed API key.** BYOK is the architecture, not a temporary stage.
- **Barcode scanning or a curated food database.** The LLM's estimate replaces the database lookup.
- **Medical-grade accuracy or clinical advice.** The coach is a motivational and planning aid.
- **Wearable / HealthKit / Google Fit sync.** Considered post-v1.1.
- **Web target.** Flutter web is technically reachable but out of scope for now.

1.4 Target user

PRIMARY PERSONA — "THE RETURNING TRACKER"

An adult who has tried MyFitnessPal or similar and quit within a month. Technically comfortable enough to paste an API key. Eats mostly home-cooked or regional food that mainstream databases cover poorly. Wants to lose or maintain weight, cares about protein, weighs themselves irregularly (monthly is normal), and will only keep logging if it takes seconds.

Core scenario: finishes lunch, opens the app, holds the record dock, says what they ate, glances at the parsed card, taps save, closes the app. Total elapsed: ~12 seconds. At month's end they open the Month tab, see a wall of green days and a downward weight line, and keep going.

2. Feature Requirements

Status legend: `BUILT` shipped and covered by tests · `PLANNED` scoped, not yet implemented.

F1 · BYOK & custom vocabulary `BUILT`

Settings → OpenAI & voice. The user's own OpenAI credential and the phrase list that makes local foods transcribe correctly.

- The API key is stored in the platform secure vault (iOS Keychain / Android EncryptedSharedPreferences) via `flutter_secure_storage` — never in SQLite, never in a backup file.
- The full key is never rendered after saving: it displays as `sk-••••••••abcd` with **no reveal control**. Changing it is an explicit "Replace key" action.
- Basic format validation before save.
- Optional comma-separated **vocabulary hints** ("Jollof, Egusi, Amala, Suya, Akara, Dodo") passed as the transcription API's `prompt` parameter to bias phonetic decoding.

F2 · Voice meal logging & macro calculation `BUILT`

Speech (or typed text) → structured food record with items, portions, calories, macros and a meal type.

REQUIREMENT	DETAIL
Audio capture	`record` package, .m4a, with a live amplitude stream driving the on-screen waveform.
Transcription	`POST /v1/audio/transcriptions`, model `gpt-transcribe` (`whisper-1` also supported), with vocabulary hints as `prompt`.
Parsing	`gpt-4o-mini` with Structured Outputs (strict JSON Schema) — see §4.2.
Typing fallback	"Type instead" on the idle record sheet runs the identical parse pipeline, skipping transcription — for noisy rooms, a broken mic, or a tired voice.
No-key onboarding	With no saved key the sheet explains what is needed and offers "Go to Settings", which closes the sheet and jumps to the Settings tab.
Extracted fields	Per item: name, quantity, kcal, protein g, carbs g, fat g. Per entry: summary, totals, and a meal type inferred from the food plus local time.
Confirm before save	A confirmation card shows parsed items, macro breakdown and a user-adjustable meal-type picker. Nothing is persisted until the user confirms.
Fix the words	From the confirm stage, "fix the words" returns to the typing view pre-filled rather than discarding the attempt.

F3 · Voice exercise logging & deterministic burn `BUILT`

The same pipeline for movement — with one hard rule: **the model extracts structure only** (activity name, sets, reps, duration). `estimated_calories_burned` is always `null` in the model's reply; the app prices every burn locally (§4.3).

- A spoken session becomes **one timeline entry** with its exercises nested as items — the mirror of a meal's food items. "5 knee press-ups and 5 dips" lands as one row titled "Knee Press-ups + 1 more".
- Burns are priced off the user's latest weigh-in. **If the weight is unknown, the burn is zero**, not an invented figure.

F4 · Daily dashboard BUILT

- **Calorie ledger:** consumed, burned, net, and **Left** (maintenance − net) — green while under budget, jollof-orange when over.
- **Period glance:** a compact readout of what is left for the current *week* and *month*, budgeted from today to period end so a late sign-in never inflates the number. Tappable through to the Month tab.
- **Macro card:** daily protein / carbs / fat totals.
- **Day's Wave (signature):** the day's entries drawn as a waveform — one peak per entry, height ∝ calories, colour by type, x-position by time of day. Tapping a peak reveals its summary; an empty day is a flatline invitation to log.
- **Timeline:** chronological feed of meals (with macro detail) and workouts for the active date, with per-entry delete.
- **Edit & re-parse:** any entry opens an editor — correct the words and re-parse with AI, or fix by hand (quantities scale calories and macros; add or remove items; change meal type). Saving updates the row **in place** (same id and timestamp) so timeline position and the day's maintenance snapshot survive. This is the fix for a mis-priced item or a forgotten side, without re-dictating.

F5 · Month calendar, maintenance colours & streaks BUILT

- **Colour coding:** a logged day is **ugu green** when net kcal ≤ that day's maintenance target, **jollof** when above, neutral when nothing was logged.
- **Per-day snapshots:** every log writes the maintenance target in force that day (`day_maintenance`); the calendar and the on-plan streak judge each day by its own snapshot, so changing the target today never rewrites history.
- **Streaks:** LOG STREAK (consecutive days with ≥1 entry) and ON-PLAN (logged *and* at/below maintenance), computed over the last 90 days.
- **Week & month ledgers:** two cards above the calendar; the week card (Mon–Sun) carries its own navigation. For an ongoing period the budget runs **today** → **period end** at each day's target, and LEFT = budget − net − overspend carried in from earlier tracked days. **Untracked days count for nothing** — skipped days never inflate LEFT, and under-eating never banks extra allowance.
- **Navigation:** tapping a day jumps the Today dashboard to that date.

F6 · Weight tracking BUILT

- **Today card:** latest weight, 30-day delta, and a hairline custom-painted trend sparkline.
- **Weigh-in sheet:** date, weight in kg *or* lb (a per-entry input choice — readings are always stored and displayed in kg, so there is no global unit setting to configure), optional note.
- **History screen:** full list newest-first with delete, plus an add FAB.
- Designed for an irregular rhythm — monthly weigh-ins are a supported use case, not a degraded one.

F7 · Coach chat with memory BUILT

A persistent conversation on the same BYOK key, fed the user's live numbers plus a distilled, user-editable memory layer.

- **Context digest** assembled per message: maintenance target, today's totals, last 7 days, weight trend, streaks, active memories, saved exercises.
- **Persisted thread** (`chat_messages`) — one conversation, with a "new conversation" reset that memories and the exercise library survive.
- **Memory distillation:** roughly every 4 exchanges, a strict-schema call extracts `{topic, content, category, action: create|update|remove}` facts. Upsert is latest-wins per topic; stale facts can be retired by the model.
- **Memory manager:** view / edit / add / delete, with a source label (you said · coach learned) and a master **"Let the coach remember"** switch that halts both writes and injection.
- **Privacy:** memories live only on-device and travel to OpenAI solely as chat context.

F8 · Auto-saved exercise recommendations BUILT

- When the coach gives exercise advice, a **keyword-gated** strict-schema extraction call pulls the concrete recommendations out (`has_exercise_advice` + name, description, muscle groups, sets/reps/rest/duration, difficulty, plan name) and auto-saves them — no hunting through chat history. The gate keeps non-exercise chats from spending an extra API call.
- **Deduped by name** (case-insensitive); duplicates are silently skipped.
- **Workouts screen:** each card supports *Copy*, *Ask* (a follow-up coach turn about that exercise), *Log it* (runs it through the parse pipeline and saves it as today's exercise entry), and remove.
- **Routines:** a named plan ("Leg Day A") groups its exercises via `plan_name`.

F9 · Gap-aware daily reminders BUILT

A local notification at a user-chosen time that only nudges when the day still has gaps.

- Settings → Daily reminder: on/off switch plus time picker. Runtime permission flow for iOS and Android 13+.
- **Inexact daily scheduling** (`inexactAllowWhileIdle`) so no exact-alarm permission is required on Android.
- **Message selection** re-evaluated at every re-arm point (launch, foreground resume, each log add/delete, settings change): nothing logged → "What did you eat today?"; a missing breakfast/lunch/dinner (snacks never count) → "Anything else today? (…)"; meals complete but no workout → "Did you get your workout in?"; a complete day → silent today, but the daily repeat is **re-armed for tomorrow, never cancelled**.
- **Survives reboot** via a boot receiver (`RECEIVE_BOOT_COMPLETED`), and the app declares `ScheduledNotificationReceiver` itself — required since `flutter_local_notifications` v16 stopped declaring its own, or scheduled alarms fire into nothing.

F10 · First-run profile & adaptive maintenance target BUILT

The maintenance target is the single line the calendar, budgets, streaks and coach all measure against — so it is *estimated from the user*, never guessed.

- **Onboarding gate:** a fresh install shows a 4-step flow (height → weight → birthday & sex → activity) with a progress bar and per-step validation. There is **no skip**. Existing installs (any log, weigh-in, or stored target) never see it.
- **Date of birth, not age** — a calendar picker; age is derived on the fly so it never goes stale.
- **Mifflin-St Jeor:** `BMR = 10·kg + 6.25·cm − 5·age + (5 | −161)`; maintenance = BMR × activity factor (5 levels, 1.2–1.9). Shown live on the final step.

- **The target IS the estimate.** Settings → Targets & profile shows it as a *display-only readout* that recalculates the instant any profile input changes, using the latest weigh-in for current weight. There is no hidden "apply" step.
- **Manual override is opt-in:** a "Set my own target" toggle reveals a number field with a "your profile suggests X" comparison and a one-tap "Back to automatic". A hand-set target **pauses the weight sync** — a weigh-in never silently overrides an explicit number.
- **The target follows your weight:** a weigh-in that moves weight by ≥ 0.5 kg (ignoring daily water wobble) recomputes the target. With ≥ 14 days between weigh-ins and ≥ 10 days of food logs it uses the *observed* estimate `avg daily intake + Δkg × 7,700 / days`, clamped to ±15% of the formula estimate so a skipped week can never halve or double it; with too little data it falls back to the formula. Logged workouts already live inside the weight trend, so they are never double-counted. The sheet reports the result: "target updated to X kcal".

F11 · Backup, restore & full reset `BUILT`

Because the app is device-local with no server copy, a reinstall — or an Android signing-key change, which forces an uninstall — would otherwise lose everything.

- **Export:** the whole database as a single JSON document (`app_settings`, `logs`, `weigh_ins`, `chat_messages`, `memories`, `exercise_recommendations`, `day_maintenance`) with a format version independent of the schema version, shared via the system share sheet as `weight-buddy-backup-YYYY-MM-DD-HHMM.json`.
- **The OpenAI key is deliberately excluded.** Writing a billable credential into a plaintext file the user then shares around would leak it; it must be re-entered by hand after a restore.
- **Import:** the file is parsed and validated before anything is written, with errors phrased for the user.
- **Delete all:** wipes every record, clears `app_settings` (profile, target, reminder, memory switch, demo marker) and deletes the key and vocabulary from the secure store, then returns to the first-run questionnaire — a state indistinguishable from a brand-new install.
- **Demo data (debug builds only):** ~14 days of tagged meals and workouts, a weigh-in trend, a coach conversation, memories and an exercise library. Idempotent via a `demo_seeded` marker; the key and targets survive.

3. User Experience

3.1 Information architecture

SURFACE	CONTAINS
Today (home)	Calorie ledger · period glance · macro card · Day's Wave · weight card · timeline. The record dock is pinned across the shell.
Month	Week & month budget ledgers · streaks · tinted calendar; a day taps through to Today.
Coach	The persistent chat thread; new-conversation reset.
Workouts	The auto-saved exercise library, grouped into named routines.
Settings	A hub that fans out to four focused pages — OpenAI & voice · Targets & profile · Daily reminder · Data (backup, demo, delete-all) — each with its own save flow, rather than one long scrolling form.

3.2 Design identity

Direction: "the pot and the plate" — a deep roasted-cocoa night kitchen, lit by the food itself: warm brown-black ground, raised bark surfaces, hairline ember separators.

TOKEN	HEX	ROLE
pot	`#221A17`	background
bone	`#F4EAD9`	text
jollof	`#E85D2F`	primary · meals · over-budget
plantain	`#F2B53C`	accent · protein
ugu	`#5CA96C`	exercise · on-plan
suya	`#C08A5A`	fat macro

Typography: IBM Plex Mono for every number (kitchen-scale readouts), Karla for reading text, small-caps mono labels. **Signature:** the Day's Wave — the day drawn as a waveform on the dashboard, echoed live on the record sheet while speaking. **Launcher icon:** the Day's Wave (jollof bars, plantain centre peak) on the pot ground.

3.3 Key flows

LOG A MEAL BY VOICE

Open app → hold the record dock → speak → release → (transcribe → parse, <4 s, waveform animates) → confirm card with items, macros, meal-type picker → *Save* → the entry appears as a new peak on the Day's Wave and a row in the timeline; the reminder re-arms with a fresh gap-aware message.

CORRECT A WRONG ESTIMATE

Tap the timeline row → details sheet → *Edit* → either fix the transcript and *re-parse with AI*, or adjust quantities / add / remove items by hand → *Save* → the row updates in place, keeping its id, timestamp and the day's maintenance snapshot.

FIRST RUN

Install → onboarding (height → weight → birthday & sex → activity), maintenance estimate shown live → shell → record sheet explains the missing key → Settings → OpenAI & voice → paste key, optionally add dish names → log.

4. Technical Specification

4.1 Stack

CONCERN	CHOICE
Framework	Flutter (Dart SDK ^3.11.5) — iOS and Android; web considered a future target
State	`flutter_riverpod` (Riverpod 3)
Persistence	SQLite via `sqflite`, offline-first — **schema v4** with `onUpgrade` migrations
Secrets	`flutter_secure_storage` — Keychain / EncryptedSharedPreferences
Audio	`record` (.m4a + live amplitude), `permission_handler`
Notifications	`flutter_local_notifications` + `timezone` + `flutter_timezone`
Networking	`http` — outbound calls go only to OpenAI
Backup I/O	`share_plus`, `file_picker`, `path_provider`
Typography	`google_fonts` (cached after first load)
App id	`com.tedif.weight_buddy` (Android applicationId/namespace + iOS bundle id)

4.2 OpenAI integration

TRANSCRIPTION

`POST https://api.openai.com/v1/audio/transcriptions` · model `gpt-transcribe` (`whisper-1` supported) · `file` = recorded audio · `prompt` = the user's dish-name hints.

STRUCTURED PARSING

Model `gpt-4o-mini` with Structured Outputs. All schemas are **fully strict-mode compliant** — every property appears in `required`, and optional fields use nullable unions (`["number","null"]`, `["string","null"]`) — so responses are guaranteed well-formed and no defensive parsing is needed.

> **Division of labour.** The model only ever *parses*: structure, portions, names. Meal calories are estimated by the model from portion sizes. Exercise calories are **never** — `estimated_calories_burned` is always `null` in its reply, and the app prices the burn locally.

```
// meal — "two plates of jollof rice with two pieces of fried chicken"
{
  "entry_type": "meal",
  "summary": "Jollof rice with 2 pieces fried chicken and fried plantain",
  "meal_type": "lunch",
  "items": [
    { "name": "Jollof Rice",    "quantity": "2 plates", "calories": 700,
      "protein_g": 14, "carbs_g": 110, "fat_g": 20 },
    { "name": "Fried Chicken",  "quantity": "2 pieces", "calories": 400,
      "protein_g": 38, "carbs_g": 0,   "fat_g": 22 }
  ],
  "total_calories": 1100, "total_protein_g": 52,
  "total_carbs_g": 110,   "total_fat_g": 42
}

// exercise — "5 knee press-ups and 5 dips"  (burn priced locally, not here)
{
  "entry_type": "exercise",
  "summary": "5 knee press-ups and 5 dips",
  "exercises": [
    { "name": "Knee Press-ups", "sets": 1, "reps": 5,
      "duration_minutes": null, "estimated_calories_burned": null },
    { "name": "Dips", "sets": 1, "reps": 5,
      "duration_minutes": null, "estimated_calories_burned": null }
  ]
}
```

The top-level `activity` / `duration_minutes` / `estimated_calories_burned` fields are a legacy shape mirroring the first array entry; the `exercises` array is the source of truth for what gets logged.

4.3 Exercise burn engine (deterministic)

Implemented in `lib/utils/exercise_math.dart` — pure math, no LLM, no I/O.

METHOD 1 — MECHANICAL WORK (REPS WIN)

```
work (J) = mass(kg) × 9.81 × vertical displacement(m) × reps
kcal     = work (J) ÷ (4,184 × 0.22)     ← 4,184 J/kcal, ~22% muscular efficiency
```

Displacement constants: dips **0.40 m** × full bodyweight · pull-ups **0.50 m** × full · push-ups **0.25 m** × ~65% (hands carry the rest) · squats & lunges **0.45 m** × full · unknown movements a conservative **0.40 m** × full.

METHOD 2 — MET ON PRECISE SECONDS (DURATION ONLY)

```
kcal = MET × 3.5 × weight(kg) ÷ 200 × duration(minutes)
```

MET defaults to **8.0** (vigorous calisthenics, Compendium of Physical Activities), with per-activity values for common cardio (running ≈ 9.8, brisk walk ≈ 4.3, cycling ≈ 7.5). Duration is priced on exact minutes — never rounded up to a minimum session block.

VERIFICATION

20 bodyweight dips at 87 kg → `87 × 9.81 × 0.4 × 20 = 6,828 J` → **≈ 7.4 kcal**, inside the 6–10 kcal sanity band (MET for a 45 s set ≈ 9.1 kcal). A single 20-rep set must never reach 20 kcal; `maxKcalPerSet` caps every set as a guard against mis-parsed reps. When bodyweight is unknown the burn is **zero**, never an invented number. Mirrored in `test/exercise_math_test.dart`.

4.4 Data model

SCHEMA	TABLES & COLUMNS
v1	`logs`: id, timestamp, type (meal/exercise), summary, calories, protein_g, carbs_g, fat_g, raw_transcript, items (JSON array — food items for meals, exercise items for workouts)
v2	`weigh_ins` (id, date, weight_kg, note) · `app_settings` KV (maintenance_kcal, reminder_enabled, reminder_time, memory_enabled, target_custom, height_cm, birthday, sex, activity_level, profile_completed) · `chat_messages` (id, role, content, created_at) · `memories` (id, topic, content, category, source, created_at, updated_at, active — unique active index on topic for latest-wins upsert) · `exercise_recommendations` (id, name, description, muscle_groups JSON, sets, reps, rest_seconds, duration_minutes, difficulty, plan_name, source_chat_id, created_at, archived)
v3	`logs` gains `meal_type` (legacy rows default to `meal`) · new `day_maintenance` (day = local-midnight millis PK → maintenance_kcal), written whenever a log is added
v4	`logs` gains nullable `sets`, `reps`, `duration_minutes` — the structured context each burn was computed from

Legacy handling: `app_settings` rows predating `target_custom` have the mode inferred; legacy `age` values are synthesised into a birthday on load. Secrets (key, vocabulary) live in the platform secure store and never enter SQLite or a backup file.

5. Non-Functional Requirements

AREA	REQUIREMENT
Latency	The two-stage pipeline (transcribe → structured parse) completes in **under 4 seconds** on a normal mobile connection.
Transcription precision	Non-Western, regional and local culinary names are handled via direct OpenAI transcription plus vocabulary hints — explicitly not native device recognizers.
Offline behaviour	The database is offline-first: viewing, editing, deleting, weigh-ins, streaks and budgets all work with no network. Only new voice/text parsing and the coach require connectivity.
Privacy	No backend, no analytics, no account. Outbound traffic goes exclusively to official OpenAI endpoints. Coach context (digest + memories + saved exercises) travels only while chatting.
Credential safety	The key never appears in the UI after saving, never enters SQLite, and never enters an exported backup.
Data durability	Full JSON export/import round-trips every table the wipe clears, so a reinstall or signing-key change is survivable.
Accessibility	Visible keyboard focus · ≥44 dp touch targets · screen-reader summaries on the Day's Wave · `prefers-reduced-motion` respected (the peak draw-in animation is skipped).
Resilience	Reminders survive reboot and app-kill via boot receiver and re-arm-on-launch. Strict schemas guarantee parseable model output.
Cost	Per-log cost is the user's own OpenAI spend; extra calls are gated (memory distillation every ~4 exchanges; exercise extraction keyword-gated).

5.1 Quality bar

Unit and widget tests cover the dashboard empty state, the record sheet's settings gate, the BYOK form, macro aggregation, add/delete persistence, the v2 schema, streaks, the coach context digest, the strict-schema OpenAI parsing and extraction methods, the Month and Coach tabs, calorie and exercise math, adaptive/smart target sync, period totals, the entry editor, backup round-trip, unit conversion and reminder logic — plus an integration test for the edit flow. `flutter test` is the gate; `tool/ship.sh` drives release builds.

6. Success Metrics

<15s	<4s	≥5/7	<10%
MEAL LOGGED, END TO END	VOICE → PARSED CARD	DAYS LOGGED PER WEEK	ENTRIES NEEDING EDIT

METRIC	DEFINITION & TARGET
Time to log	App open → entry saved. Target < 15 s for a voice meal.
Pipeline latency	Release of the record button → confirm card rendered. Target < 4 s.
Logging adherence	Days with ≥1 entry per week. Target ≥ 5/7 sustained past week 4 — the point where mainstream trackers lose the user.
Parse quality	Share of saved entries subsequently edited. Target < 10%; a rising number points at either transcription or portion estimation.
Transcription accuracy	Regional dish names correctly transcribed on a fixed spoken test set, with and without vocabulary hints.
Target trustworthiness	Share of users still on the automatic target (rather than a manual override) after 30 days.
Data safety	Zero reports of a lost history after reinstall; zero credential exposure incidents.

Note: with no backend and no analytics by design, these are measured through the owner's own use and manual test passes, not telemetry. That is an accepted consequence of the privacy stance.

7. Risks & Mitigations

RISK	MITIGATION
BYOK is a hard onboarding step. Obtaining and pasting an OpenAI key filters out most mainstream users.	Accepted — it is what makes zero-backend and zero-subscription possible. Mitigated by a clear no-key state that routes straight to Settings, and by the app being fully usable offline for viewing and editing once data exists.
LLM calorie estimates are approximate. Portion inference from "two plates" carries real error.	Confirm-before-save, in-place editing with re-parse, and quantity scaling. The adaptive target additionally self-corrects against observed weight change, so systematic estimation bias washes out of the number that matters.
Model or API drift. A deprecated model or changed Structured Outputs behaviour breaks parsing.	Strict schemas with full `required` coverage; both `gpt-transcribe` and `whisper-1` supported; parsing isolated behind `openai_service.dart` with direct tests.
Device loss or forced uninstall. No server copy exists.	F11 JSON export/import covering every wiped table, with a versioned format independent of the schema.
Android reminder fragility. OEM battery managers and the v16 receiver change silently kill schedules.	App-declared `ScheduledNotificationReceiver`, boot receiver, inexact scheduling (no exact-alarm permission), and re-arm on every launch, resume and log mutation.
Runaway API cost. Chat plus per-log calls are billed to the user.	Keyword-gated exercise extraction, memory distillation only every ~4 exchanges, `gpt-4o-mini` for parsing, and no background polling.
Coach overreach. A chat model giving nutrition advice can produce unsafe suggestions.	Positioned as motivational and planning support, not clinical advice; all numbers it reasons over are the user's own, and the memory layer is fully user-editable and switchable.

8. Release Status & Roadmap

MILESTONE	STATUS	SCOPE
v1.0.4 current	`SHIPPED`	F1–F11 complete: voice + typed logging, macros, deterministic burn engine, dashboard with the Day's Wave, month calendar & streaks, weight tracking with adaptive target sync, coach chat with memory, auto-saved exercise library, gap-aware reminders, first-run profile, backup/restore/reset. Schema v4.
v1.1	`NEXT`	Hardening the edit and backup flows in real use; broadening the integration-test suite beyond the edit flow; expanded regional vocabulary defaults; per-item nutrition confidence surfaced in the UI.
Later	`CONSIDERED`	Flutter web target · HealthKit / Google Fit import for steps and workouts · photo-assisted portion estimation · alternative model providers behind the same BYOK abstraction.

8.1 Open questions

- Should meal-calorie estimation also self-correct per food item (learning that *this* user's "plate of jollof" runs ~15% under the model's default), or is the target-level adaptive correction in F10 sufficient?
- Does the coach warrant a scheduled proactive nudge (weekly review), or does that conflict with the no-background-work, cost-conscious stance?
- Is encrypted backup worth adding so the key *could* travel, or is hand re-entry the right permanent trade?